

Sistema de Monitoramento de Riscos Ambientais com Grafos e Estruturas de Dados

FIAP — Global Solution 2026 | 1º Semestre | Dynamic Programming | Prof. André Marques

1. IDENTIFICAÇÃO DO GRUPO E CONTEXTUALIZAÇÃO

RM	Nome
566223	Davi Marques de Andrade Munhoz
562559	Diogo Oliveira Lima
566539	Leandro Simoneli da Silva
562414	Lucas dos Reis Aquino
565356	Lucas Perez Bonato

Grupo TechVerse. O Brasil figura entre os países mais vulneráveis às mudanças climáticas. As enchentes no Rio Grande do Sul (maio/2024) afetaram **478 municípios**, deslocaram >500 mil pessoas e causaram prejuízos >R\$60 bi. Paralelamente, o **MATOPIBA** concentra municípios com crescente risco de seca impulsionado pelo agronegócio e redução de chuvas. Satélites GOES-16 e Sentinel transmitem dados em tempo real de temperatura, precipitação e cobertura vegetal, gerando redes geoespaciais complexas. Foram selecionados dois cenários: **Cenário A** — rede de resposta a enchentes no RS (N=12 municípios, 15 arestas) — e **Cenário B** — triagem de seca no MATOPIBA (N=10, 14 arestas). Alinhamento: ODS 2, 9, 11 e 13 da ONU.

2. MODELAGEM: GRAFO, BST E ESTRUTURAS DE DADOS

Grafo $G = (V, E)$

Representado como dicionário de listas de adjacência: $adj = \{id: [(vizinho, peso), ...]]$. Cada vértice é uma tupla imutável (id, nome, índice_risco, custo_atendimento, população). Pesos = tempo de deslocamento (h) no Cenário A e distância rodoviária (km) no Cenário B. Justificativa frente à matriz: grafos esparsos (2–5 vizinhos/vértice) — lista ocupa $O(V+E)$ vs. $O(V^2)$ da matriz; com $V=478$ a matriz exigiria ~900 KB, a lista ~10 KB.

BST (Binary Search Tree)

Chave = índice de risco (float), propriedade $r_{esq} < r_{pai} \leq r_{dir}$. Implementada em Python puro (classes Node e BinarySearchTree) com operações: inserir, buscar(r_{min} , r_{max}), percurso_in_order, altura e remover. Cenário A: altura 5 (fator 1,25 sobre $\log_2 13 \approx 4$ — bem balanceada). Cenário B: altura 7 (fator 1,75 — moderadamente desbalanceada). O percurso in-order retorna municípios em ordem crescente de risco; a busca por intervalo opera em $O(k + \log n)$ — eficiente para filtrar municípios críticos em tempo real.

Estrutura	Uso no Sistema	Complexidade
Tupla	Vértice imutável (id, nome, risco, custo, pop)	$O(1)$ acesso
Lista	Adjacência; resultado de caminhos	$O(\text{grau})$ iteração
Dicionário	Adjacência; dist/pred no Dijkstra	$O(1)$ médio
Conjunto	Visitados em BFS/DFS; fronteira Guloso	$O(1)$ pertencimento
Heap (heapq)	Fila de prioridade mínima no Dijkstra	$O(\log V)$ push/pop
BST	Municípios por risco; busca intervalar	$O(\log n)$ / $O(k + \log n)$
Deque	Fila FIFO para BFS	$O(1)$ append/popleft

3. ANÁLISE DE COMPLEXIDADE TEÓRICA

Aspecto	Força Bruta (Backtracking)	Dijkstra (Guloso)
Tempo (pior caso)	$O(V!)$ — todos os caminhos hamiltonianos	$O((V+E) \log V)$ — heap binário
Tempo (prático)	$O(d^V)$, d = grau médio	Sub-ms até $N=100$
Espaço (pilha/heap)	$O(V)$ — profundidade máxima da recursão	$O(V)$ — dist[], pred[], heap
Espaço (caminhos)	$O(k \cdot V)$, k = caminhos armazenados	—
Limite prático	$N \leq 10-12$ ($N=10$: ~109k caminhos, 213 ms)	$N \leq 10.000$ em produção
Resultado RS	12 vértices: inviável em produção	30 relax., 12 vért., 0,02 ms

Força Bruta: a cada chamada itera sobre vizinhos não visitados acumulando custo. Contador global registra chamadas recursivas e caminhos avaliados — papel de oráculo validador. **Dijkstra:** extrai o vértice de menor distância acumulada via heap mínimo e relaxa suas arestas. A escolha local — processar sempre o vértice mais próximo não finalizado — é suficiente para garantir otimalidade em grafos com pesos não-negativos (prova por contradição).

4. RESULTADOS

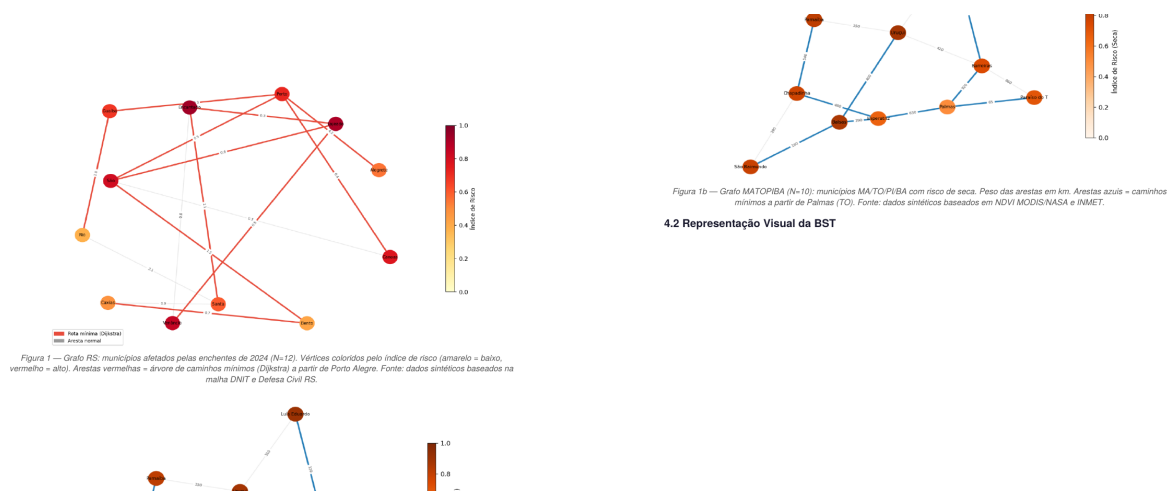


Fig. 1 — Grafo RS (N=12, arestas vermelhas = Dijkstra de Porto Alegre) | Fig. 1b — Grafo MATOPIBA (N=10, arestas azuis = Dijkstra de Palmas).

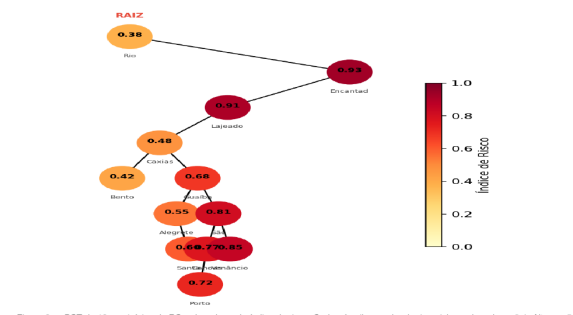


Figura 2 — BST de 12 municípios do RS ordenados pelo índice de risco. Cada nó exibe o valor do risco (chave de ordenação). Altura = 5, fator 1,25 sobre o ideal $\log_2(13) \approx 4$ — considerada bem balanceada. A cor dos nós segue a escala de risco: amarelo (baixo) → vermelho (alto). Fonte: elaboração própria com matplotlib.

4.3 Plano de Atendimento — Dijkstra + BST (Cenário A)
A BST consultou municípios com risco $\geq 0,77$. O Dijkstra calculou a rota mínima a partir de Porto Alegre (hub de defesa civil) para cada um deles. Resultado com 30 relaxamentos, 12 inserções e 12 extrações do heap:

Município	Risco	Dist. (h)	Rota Mínima
Canoas	0,77	0,40	Porto Alegre → Canoas
São Leopoldo	0,81	0,50	Porto Alegre → São Leopoldo
Lajeado	0,91	1,30	Porto Alegre → São Leopoldo → Lajeado
Encantado	0,93	1,60	Porto Alegre → São Leopoldo → Lajeado → Encantado
Venâncio Aires	0,85	1,80	Porto Alegre → São Leopoldo → Lajeado → Venâncio Aires

Fig. 2 — BST dos 12 municípios do RS (chave = índice de risco). Altura = 5, fator 1,25 sobre $\log_2 13 \approx 4$.

Plano de Atendimento — Dijkstra + BST (Cenário A, risco $\geq 0,77$)

Município	Risco	Dist. (h)	Rota Mínima
Canoas	0,77	0,40	Porto Alegre → Canoas
São Leopoldo	0,81	0,50	Porto Alegre → São Leopoldo
Lajeado	0,91	1,30	Porto Alegre → São Leopoldo → Lajeado
Encantado	0,93	1,60	Porto Alegre → São Leopoldo → Lajeado → Encantado
Venâncio Aires	0,85	1,80	Porto Alegre → São Leopoldo → Lajeado → Venâncio Aires

5. ESCALA DE DECISÃO E GAP DE OTIMIZAÇÃO

4.4 Desempenho Comparativo: Tempo × N

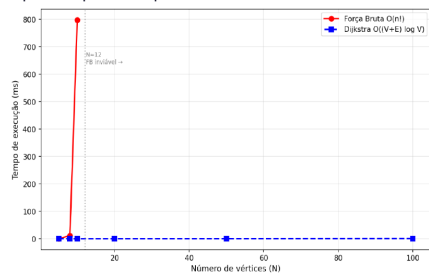


Figura 3 — Tempo de execução × N para Força Bruta e Dijkstra. Escala linear. A Força Bruta cresce fatorialmente: $N=4 \rightarrow 10$ ms, $N=10 \rightarrow 1$ s. O Dijkstra permanece sub-milsegundo até $N=100$. O cruzamento das curvas ocorre próximo a $N=7$, ponto a partir do qual a FB se torna inviável para uso em produção. Fonte: benchmark com grafos sintéticos.

4.5 Gap de Otimidade: Força Bruta × Dijkstra



Fig. 3 — Tempo × N (Força Bruta cresce fatorialmente; Dijkstra permanece sub-ms até $N=100$). Fig. 5 — Gap de otimalidade = 0% em todos os tamanhos ($N=4-10$); FB: $N=10 \rightarrow 330$ ms vs. 0,05 ms do Dijkstra (razão ~6.600×).

Nível	Algoritmo	N Viável	Qualidade	Custo Comp.	Recomendação
1 — Ótimo	Força Bruta	$N \leq 8$	100% global	$O(V!)$	Validação/testes
2 — Ótimo	Dijkstra	$N \leq 10.000$	~100% (gap=0%)	$O((V+E)\log V)$	✓ Produção
3 — Bom	Dijkstra heurístico	$N \leq 100k$	Alto	Muito baixo	Grande escala
4 — Inviável	Força Bruta	$N > 12$	100% teórico	Explosão fat.	✗ Nunca

N	Custo FB	Custo Dijkstra	Gap (%)	Tempo FB (ms)	Tempo Dij (ms)
4	3,170	3,170	0,00%	0,022	0,019
5	1,310	1,310	0,00%	0,025	0,012
6	4,010	4,010	0,00%	0,064	0,013
7	6,300	6,300	0,00%	0,330	0,014
8	1,980	1,980	0,00%	2,526	0,019
9	2,650	2,650	0,00%	35,923	0,029
10	3,010	3,010	0,00%	330,421	0,050

O gap de otimalidade foi **0,00%** em todos os tamanhos testados. O Dijkstra é matematicamente ótimo para grafos com pesos não-negativos — propriedade válida nos dois cenários (tempos e distâncias sempre positivos). A FB serve como oráculo de validação para instâncias pequenas; torna-se inviável em $N \geq 11$ (>5 s) e impraticável para os 478 municípios reais do RS. A lista de adjacência reduziu espaço de $O(V^2) \rightarrow O(V+E)$; o heap garantiu $O(\log V)$ por relaxamento; a BST permitiu consultas por intervalo em $O(k+\log n)$ vs. $O(n)$ na varredura linear.

6. CONCLUSÃO E CONEXÃO COM ODS

O sistema demonstra que algoritmos fundamentais aplicados a estruturas adequadas resolvem problemas de alta relevância social com desempenho de produção. O Dijkstra integrado à BST calcula rotas ótimas em **<1 ms** para até 100 municípios, com gap de otimalidade de 0% frente à Força Bruta. A FB demonstrou empiricamente a explosão combinatória: caminhos crescem de 65 ($N=6$) a 109.601 ($N=10$). **Recomendação prática:** para o sistema de Defesa Civil do RS com 478 municípios, adotar Dijkstra + lista de adjacência + BST para filtragem de alto risco + heapq como fila de prioridade. A complexidade $O((V+E) \log V)$ garante resposta em tempo real na escala completa do estado. **Conexão ODS:** ODS 2 (Fome Zero — cooperativas MATOPIBA), ODS 9 (Infraestrutura — otimização de rotas), ODS 11 (Cidades Sustentáveis — resposta a desastres no RS) e ODS 13 (Ação Climática — monitoramento com dados satelitais GOES-16/Sentinel).

7. REFERÊNCIAS

- CORMEN, T. et al. *Introduction to Algorithms*, 4ª ed. MIT Press, 2022. Caps. 22–25.
- SEDGEWICK, R.; WAYNE, K. *Algorithms*, 4ª ed. Addison-Wesley, 2011. Parte 4: Grafos.
- SKIENA, S. *The Algorithm Design Manual*, 3ª ed. Springer, 2020.
- IBGE. Malha municipal brasileira. Disponível em: ibge.gov.br/geociencias
- INMET. Banco de Dados Meteorológicos. Disponível em: bdmep.inmet.gov.br
- DNIT. Malha viária federal (shapefile). Disponível em: dnit.gov.br
- NASA Earthdata. MODIS NDVI MOD13A3. Disponível em: earthdata.nasa.gov
- Defesa Civil RS. *Relatório Enchentes 2024*. Porto Alegre, 2024.
- Python Software Foundation. heapq — Heap queue algorithm. docs.python.org/3/library/heapq.html
- Python Software Foundation. tracemalloc — Trace memory allocations. docs.python.org/3
- Repositório: github.com/TechVerseFiap/GS.SpaceConnect.Dynamic